Lec 03: Kernel mode of execution

Graduate Systems (CSE638)

Instructor: Rinku Shah

References:
- Operating Systems: Three Easy Pieces, Remzi & Andrea
https://pages.cs.wisc.edu/~remzi/OSTEP/
- Design and Engineering of Computer Systems, Mythili Vutukuru (IITB)
https://www.cse.iitb.ac.in/~mythili/decs/

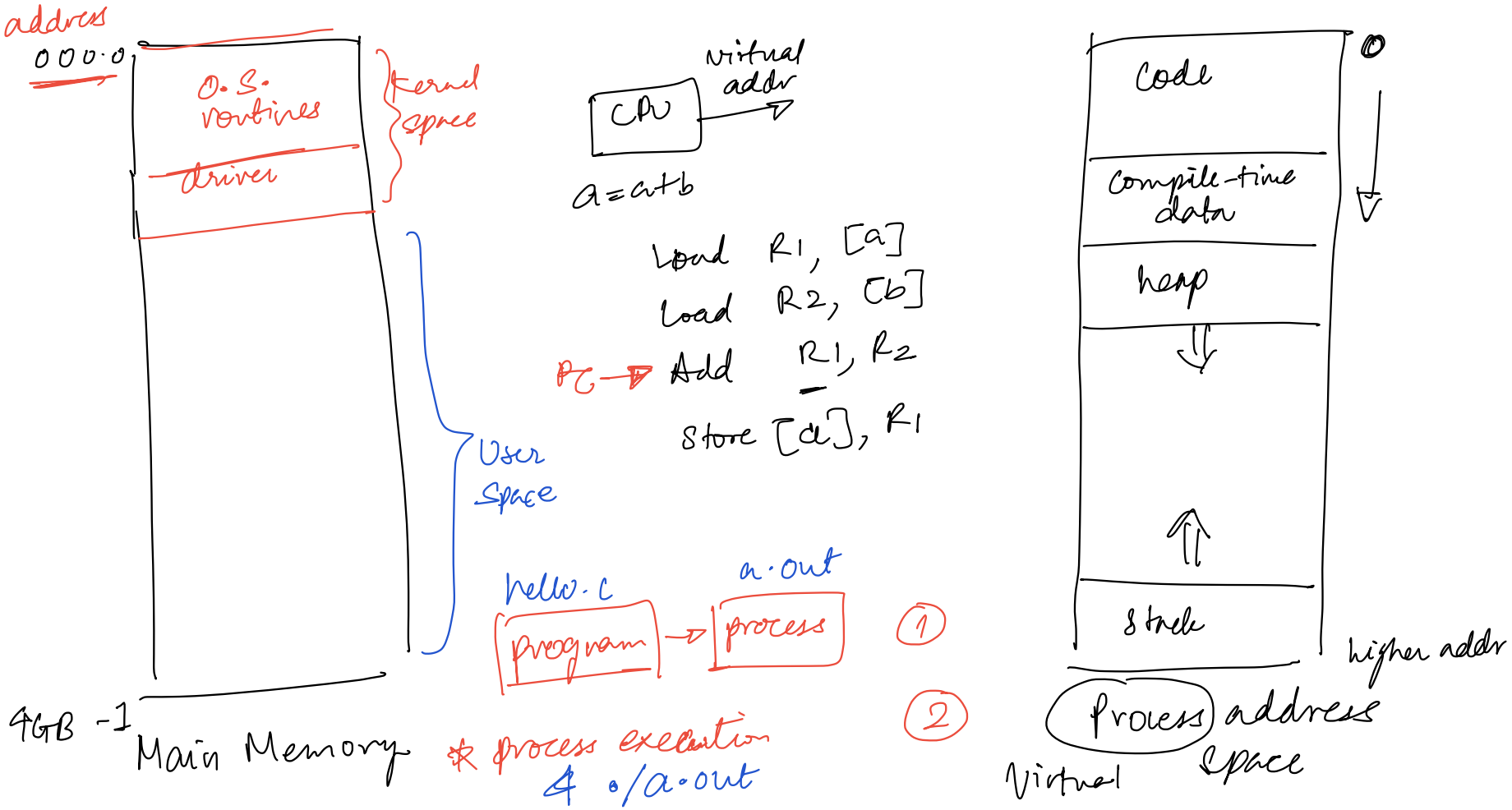


INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY
DELHI

The Process

Process management tasks by OS

- Process lifecycle management
- Scheduling policy
- **Context switching**
- Implement system call API
- **Handle interrupts and other events**


address
0000
O.S. routines
driver
Kernel space
User Space
4GB -1
Main Memory
CPU
virtual addr
a = a+b
Load R1, [a]
Load R2, [b]
PC → Add R1, R2
Store [c], R1
hello.c
program
a.out
process
1
2
* process execution
& ./a.out
Code
compile-time data
heap
Stack
higher addr
Process address space
Virtual

fork

parent process

↓

replica of process image

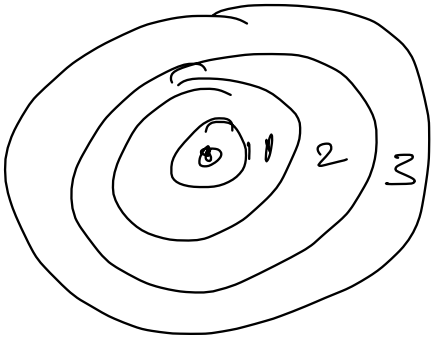
↳ process image of child

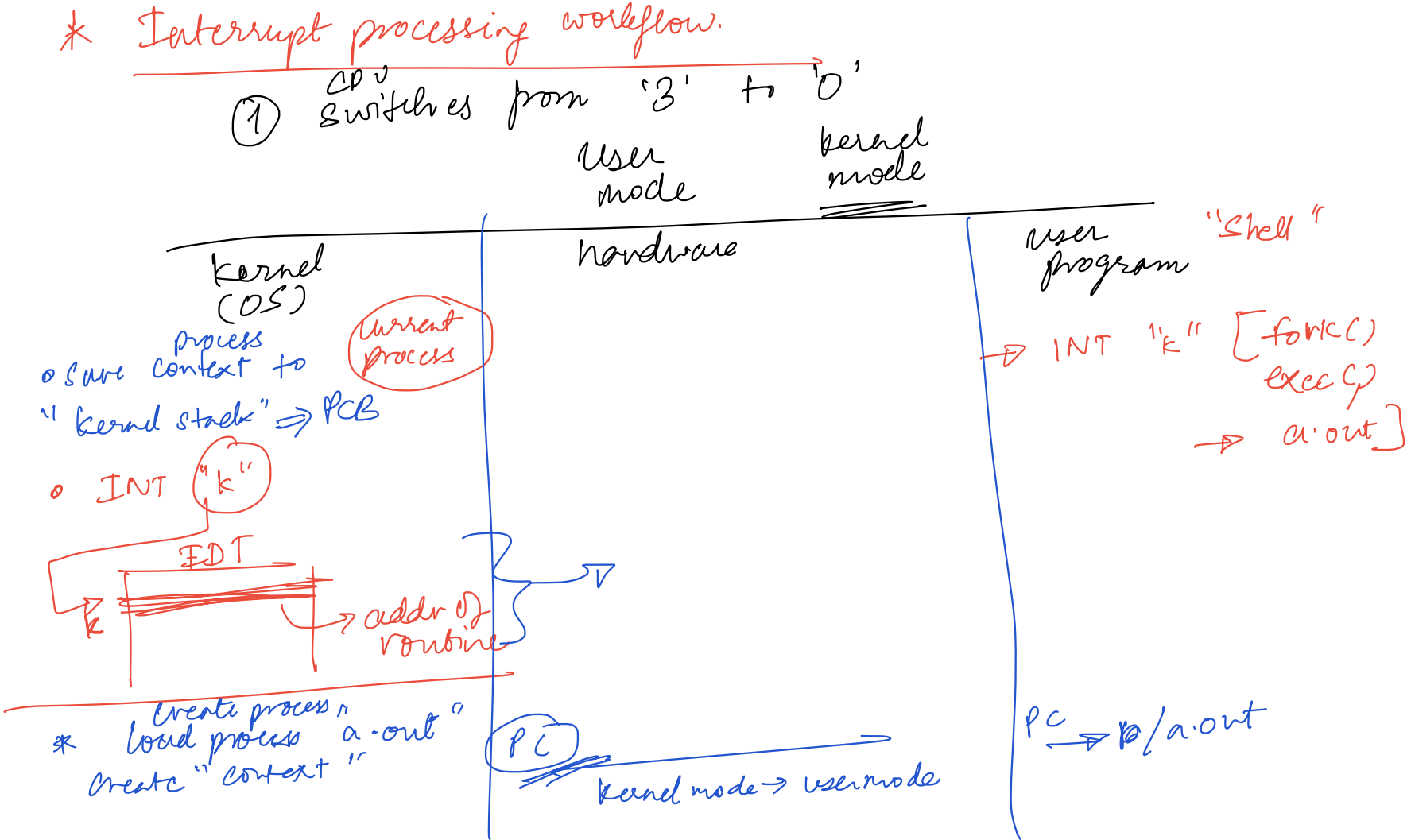
X86

* privilege levels ⇒ 0, 1, 2, 3

0 → highest privilege level → OS

3 → lowest privilege level → user prog




* Interrupt processing workflow.
① CPU switches from '3' to '0'
user mode
kernel mode
kernel (OS)
hardware
user program
"shell"
process
save context to "kernel stack" ⇒ PCB
current process
INT "k"
IDT
k
addr of routine
create process
load process "a.out"
create "context"
PC
kernel mode → user mode
INT "k" [fork()
execv()
→ a.out]
PC → p/a.out

Tutorial Recap: Process life cycle (fork, exec, exit, wait)

```
…
int ret = fork();
//fork returns “child pid” to parent
if(ret==0){
        printf(“Child process\n”);
}
else if(ret>0){
        printf(“Parent process\n”);
}
…
```

```
…
int ret = fork();
//fork returns “child pid” to parent
if(ret==0){
        exec(<some_executable>);
        printf(“error: exec failed\n”);
}
else if(ret>0){
        printf(“Parent process\n”);
}
…
```

parent

```
…
int ret = fork();
//fork returns “child pid” to parent
if(ret==0){
        printf(“Child process\n”);
        exit();
}
else if(ret>0){
        printf(“Parent process\n”);
        wait();
}
…
```

exec("./a.out"

child

Process

- Unique Process Identifier (PID)
- Memory image
- Execution context of the process
  - PC
  - CPU register values when process is running
  - Context is saved in memory when the process is paused, restored when run again
  - Communication with I/O
    - List of open files
    - Active connections, etc.

Process Control Block (PCB)

All information about a process is stored in a data structure called **Process Control Block (PCB)**

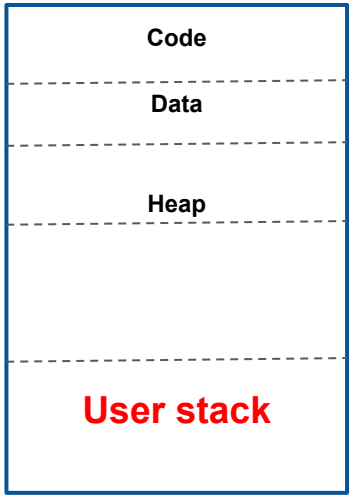
- Process Identifier (PID)
- Process state (running, ready, blocked, terminated, etc.)
- Pointers to related processes (parent, children)
- Saved CPU context of process when it is not running
- Information related to memory locations of a process
- Information related to ongoing I/O communication
- …
- OS stores PCB's of active processes in a data structure (array, lst, …)
    - New PCB added when process is created, deleted when process is cleaned up

How does a process implement a function call?

Function call

- User program → Function call
  - Allocate memory on user stack for function arguments, local variables
  - Push return address (PC) → PC jumps to function code
  - Push register context (to resume execution when function returns)
  - Execute function
  - On return from function → Pop return address; Pop register context

```
int c;
int mult(int a, int b) {
        int temp;
        temp = a * b;
                return temp;
}
main() {
        int a = 100;
        int b = 50;
        c = mult(a, b);
        int *p = malloc(500);
}
```



What are system calls? How do they work?

OS runs a process → Direct execution – What is the problem?

- Process is created (fork())
  - OS adds new PCB to list, copies memory image of parent to the child
- OS scheduler loops over ready processes, periodically → Timer interrupt
  - Find a process to run next
  - Save the context of existing process; restore context of new process
- Once the process is context switched in, OS is out of picture, CPU in user mode, run user code directly

OS	Program
Create entry for process list	
Allocate memory for program	
Load program into memory	
Set up stack with argc/argv	
Clear registers	
Execute **call** main()	
	Run main()
	Execute **return** from main
Free memory of process	
Remove from process list	

Direct Process execution – What is the problem?

OS	Program
Create entry for process list	
Allocate memory for program	
Load program into memory	
Set up stack with argc/argv	
Clear registers	
Execute **call** main()	
	Run main()
	Execute **return** from main
Free memory of process	
Remove from process list	

Problem#1: Restricted operations
Ex. Privileged resource access (e.g., hardware) requires protection

Solution: Multiple privilege levels; System calls

User mode vs. Kernel mode of a process

- User code in execution → CPU (user mode)
- When does CPU transition mode? **User mode → Kernel mode**
  - Process makes a **system call**
  - External device raises an **interrupt**
  - **Fault occurs** during program execution
- Such events are called **“TRAPS”**
  - **CPU traps into OS code**
  - **CPU → kernel mode → runs OS code → returns to user code (user mode)**

User mode vs. Kernel mode of a process

- **Process P → Kernel mode – to run OS code**
  - **Still Process P is in RUNNING state**
- **OS is not a SEPARATE process**
  - **OS runs in kernel mode of existing processes**

System call vs. Function call — SYSTEM CALL

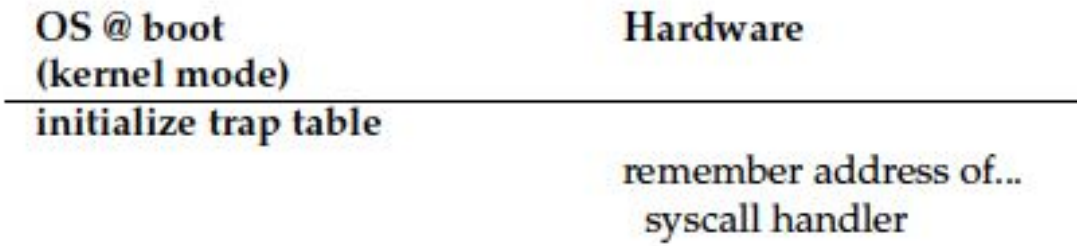
- Common with System call
  - Use of stack for push/pop register context
  - Save old PC; change PC to point to OS code to handle system call
- Changing PC
  - Function call – Directly jump to function code address (“call” in x86)
  - System call – Cannot trust user to jump to correct OS code
- Saving register context on stack
  - Function call – saved & restored from user stack
  - System call – cannot trust user stack; what if user has setup malicious values on stack
- Need for a **secure stack**, and a **secure way of jumping to OS code**
  - Solution: **Kernel stack** & **Interrupt Descriptor Table (IDT)**

System call – Kernel stack

- Every process uses a **separate kernel stack** for **running kernel code**
  - Part of PCB of process, in OS memory
  - Only accessible in kernel mode
  - Used like user stack, but for kernel mode execution
  - In kernel mode
    - Context pushed on kernel stack during system call, popped on return

System call – Kernel stack & IDT

- To set PC, CPU accesses **Interrupt Descriptor Table (IDT)**
  - Data structure with addresses of kernel code to jump for events
  - Set up by OS in memory, not accessible in user mode
  - CPU uses IDT to locate the OS code location to jump to



Secure stack for OS to run + Secure way to locate OS code

Hardware trap instruction

- User code → system call; use special **trap instruction**
  - **Ex.:** “int n” in x86 where “n” indicates type of trap (syscall, interrupt)
  - The value of “n” specifies index into IDT array
    - has address of which OS function to jump to
- When CPU runs the trap instruction
  - CPU moves to higher privilege level (ring 0)
  - CPU shifts SP(Stack pointer) register to point to kernel stack
  - PC ← OS code address from IDT index
  - Register context is saved on kernel stack (part of PCB)
  - OS code starts to run on secure stack

Why “trap” instruction?

- Why?
  - User cannot be trusted
    - to jump to correct code
    - To run OS code safely
- Who calls **trap** instruction?
  - System call code in library
    - printf invokes system call via “int n”
  - External hardware raises an interrupt
- **return from trap** instruction executed
  - Switch CPU to lower privilege level
  - Pop register context
  - Jump back to user code and user stack

Typical call flow: Limited direct execution protocol

OS @ run (kernel mode)	Hardware	Program (user mode)
Create entry for process list		
Allocate memory for program		
Load program into memory		
Setup user stack with argv		
Fill kernel stack with reg/PC		
return-from-trap		
	restore regs	
	(from kernel stack)	
	move to user mode	
	jump to main	
		Run main()
		...
		Call system call
		trap into OS
	save regs	
	(to kernel stack)	
	move to kernel mode	
	jump to trap handler	
Handle trap		
Do work of syscall		
return-from-trap		
	restore regs	
	(from kernel stack)	
	move to user mode	
	jump to PC after trap	
		...
		return from main
		trap (via `exit()`)
Free memory of process		
Remove from process list		

Problem#2:

How does OS regain control of the CPU so that it can switch between processes?

“Context switching”

Context switching

- Process makes system call, runs OS code, returns to **same user process**
  - Process made a **non-blocking** system call
- OS **cannot return to the same user process** after trap
  - “exit” sys call
  - Blocking sys call
- OS **does not want to return to the same process** after trap
  - Process has run for too long, OS wants to share CPU with others
- Process in **kernel mode invokes scheduler**
  - Identify ready process
  - Save context of currently running process in its PCB/ kernel stack
  - Restore context of new process from its PCB/kernel stack
  - Return to user mode of new process

Mechanism of context switch

- Context switch
  - Switch CPU context from kernel mode of old process (say, P1) to kernel mode of new process (say, P2)
- Before Context switch

- Mechanism of Context switch

- After Context switch

The “timer” interrupt

OS @ boot (kernel mode)	Hardware
initialize trap table	
	remember addresses of...
	syscall handler
	timer handler
start interrupt timer	
	start timer
	interrupt CPU in X ms

OS @ run (kernel mode)	Hardware	Program (user mode)
		Process A
	→ switch to "0"	...
	timer interrupt	
	save regs(A) → k-stack(A)	
	move to kernel mode	
	jump to trap handler	
Handle the trap		
Call `switch()` routine		
save regs(A) → proc_t(A)		
restore regs(B) ← proc_t(B)		
switch to k-stack(B)		
return-from-trap (into B)		
	restore regs(B) ← k-stack(B)	
	move to user mode	
	jump to B's PC	
		Process B
		...

Understand saving and restoring context

Where is the CPU context saved/restored for following scenarios?

- Process P1 has made a blocking system call and moved to kernel mode
- After handling system call, OS decides to some other process, P2. P1 cannot continue now.
- P1 becomes ready and is run by scheduler again in future
- P1 returns from trap in user mode

Homework

Find out the number of context switches undergone by your process.

Hint: The files under /proc in Linux will show this information

ps -ax | grep "a.out"

Course plan: OS fundamentals

Lecture Number	Topic
1	Introduction & System design principles
2	Program, CPU hardware, Process
3	Kernel mode of execution
4	Threads, Race condition, CS, locks (Lec 8)
5	Virtual machines & Containers (Lec 10)
6	CPU & Memory management for Virtual Machines (Lec 20)

Course plan: Networking fundamentals

Lecture Number	Topic
7	Socket programming (event-driven too)
8	Network Performance analysis & end-to-end delays
9	IP addressing, subnetting, routing
10	TCP, UDP
11	HTTP, SCTP
12	End-to-end network example including NAT, firewall, LB
13	SDN concept
14	Serialization, gRPC

What will you learn in this course?

Week Number	Lecture Topic
1, 2, 3	Principles of computer system design, Process, Threads, Virtualization (VMs, containers)
4, 5, 6	Refresh computer networks basics, SDN, TCP, UDP, SCTP, HTTP, serialisation, gRPC
7, 8	Multi-threaded server design: synchronization, atomicity, deadlocks, event-driven server; Multi-tier application design: Storage options (RDBMS, NoSQL), API design (REST, RPC, publish-subscribe); End-to-end application design (video streaming systems, etc.)
9	Deployment of computer systems: cloud deployment, cloud orchestration, network architecture (firewall, DMZ); Performance measurement and tuning: performance metrics, load testing, performance profiling
10, 11	Optimizations: caching in computer systems; Scalability: multicore speedup, load balancer design; Reliability and fault-tolerance: replication and consistency, atomicity
12, 13	Distributed transactions: sharding, consistent hashing, 2-phase commit; Putting all pieces together: Case studies of distributed systems design